

How LLMs Process Our Requests — Part 4

Choosing, Customising, and Trusting an LLM

The final part of the series.

Parts 1–3 answered **how it works**. From your keystroke through the data centre, through attention and probability and sampling, all the way out to a real assistant with tools, RAG, and guardrails.

Part 4 is about **how to make decisions about it** — the questions a business audience actually has to answer:

- "Should we fine-tune a model on our data, or just use prompts?"
- "What does it mean when a model has 'thinking' mode?"
- "Can it read this PDF I just uploaded?"
- "Does Anthropic train on our client data?"
- "How do we know the answers are any good?"

Same running example throughout the series — *"What is my portfolio's exposure to emerging-market equities?"* — but now we're looking at the system around it, not the mechanics inside it.

The Knowledge Cutoff: Why the Model Doesn't Know What Happened Yesterday

Before anything else, a foundational point that explains a whole category of model mistakes.

Every LLM was trained on a fixed snapshot of text, finalised on a specific date. That date is called the **knowledge cutoff** — sometimes also called the *training cutoff*.

If the model was trained on text up to January 2025, then for the model:

The world ends in January 2025

It has never heard of anything that happened after that date.

No sense of today's date

It doesn't know today's date unless someone explicitly tells it.

Stale on current events

No latest election results, last quarter's earnings, current Fed rate, or yesterday's market close.

Why the Cutoff Matters for Our Example

⚠ If you ask the model *"What's been driving EM equities recently?"* and the cutoff was 14 months ago, the model will confidently answer with what was happening 14 months ago — phrased as if it were current. The words will sound right. The facts will be stale.

This is the single most common source of "the model is wrong" moments. And it's **why** the techniques from Part 3 — tool use, RAG, web search — are not optional luxuries. They are the only way to get the model out of its frozen world and into the present.

Callback to Part 1, Slide 6: *"Why does it sometimes give a wrong answer with complete confidence?"* The cutoff is half the answer. The model isn't lying — it just doesn't know its information is out of date, because to it, that information is the latest there is.

Reasoning Models: The "Thinking" Mode

A shift in how the newest models work

The picture from Part 2 was straightforward: the model predicts one word, you see it streamed, repeat. That picture is accurate for most LLMs you've used.

But starting in late 2024, a new category appeared: **reasoning models**. OpenAI's o1 and o3, Anthropic's Claude with **extended thinking**, DeepSeek's R1, Google's Gemini with **deep think**. These models work differently in one important way.

1

Standard Models

Predict one token at a time, stream directly to the user. Fast, transparent, and suitable for most everyday tasks.

2

Reasoning Models

Generate a long stream of **hidden tokens** — internal "thinking" — before producing the visible answer. Slower but dramatically more capable on hard problems.

What Reasoning Models Actually Do

Before producing the answer the user sees, a reasoning model generates a long stream of **hidden tokens** — its own internal "thinking" — that the user never gets shown. Only after this hidden thinking is complete does the model produce the visible answer.

Imagine you asked a colleague a hard question and they said *"Hmm, let me think for a moment."* Then they sat there silently for thirty seconds, working it out in their head, before speaking. That's what a reasoning model is doing. The thinking is real work — real tokens being generated — it just isn't shown to you.



The hidden thinking phase is genuine computation — not a pause, but active token generation that shapes the quality of the final answer.

Why This Works

It turns out that LLMs are much better at hard problems when they're allowed to "talk to themselves" first. By generating reasoning tokens, the model effectively gives itself a scratchpad.

 Example reasoning trace: *"Let me check the holdings. EM equities means stocks in developing markets. The Vanguard FTSE EM ETF qualifies. The iShares EM bond fund does NOT, because it's bonds. Let me recompute..."* — each reasoning token sits in the context window and helps shape what comes next.

It's the same model architecture as before. Same attention, same next-word prediction. The only change is that the model has been **trained, during instruction tuning, to spend its first chunk of output on reasoning before giving the user-facing answer.**

The Tradeoffs of Reasoning Models

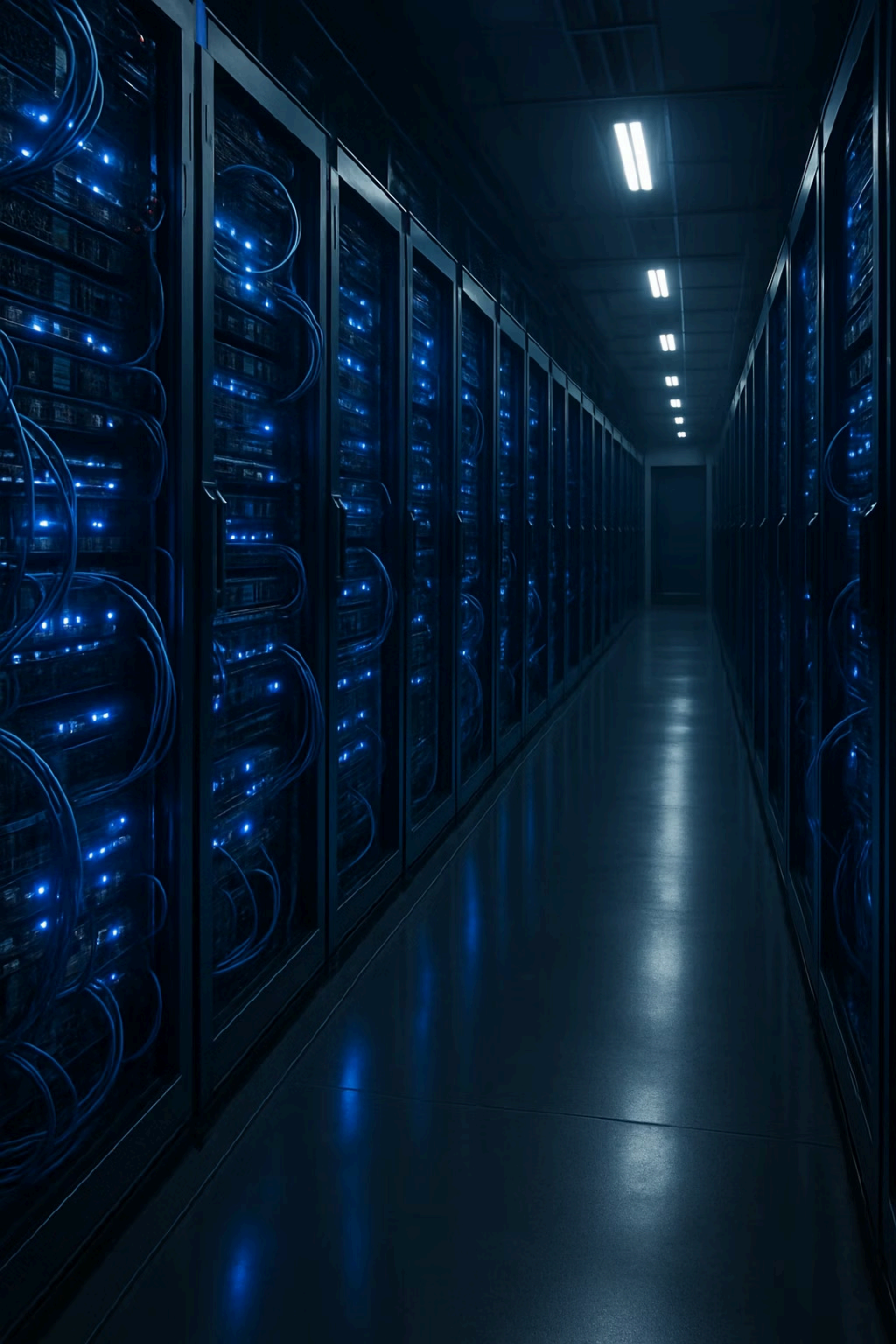
Where They Shine

- Mathematics and complex calculations
- Code generation and debugging
- Multi-step logic and planning
- Complex analysis and edge cases

The Costs

- **Much slower** — hidden thinking may take 10–60 seconds before the first user-visible word appears
- **More expensive** — hidden tokens are billed at output rates; a reasoning answer might consume 5,000–50,000 hidden tokens to produce a 200-token visible answer
- **Overkill for simple tasks** — asking "What's the capital of France?" wastes time and money

- ✓ **The practical rule:** Use reasoning models for hard problems that benefit from careful thinking (code, math, planning, complex analysis), and regular models for everyday tasks (summarisation, drafting, retrieval, conversation). Most modern products let you switch, or do it automatically.



What This Means for the Part 2 Picture

The streaming-one-word-at-a-time picture from Part 2 is still correct mechanically — reasoning models are doing the same token-by-token prediction.

What stays the same

The model is producing tokens the whole time. Same attention mechanism, same next-word prediction, same architecture underneath.

What changes

The difference is purely about **which tokens get shown to the user**. The system hides the first batch — the reasoning tokens — and only surfaces the final answer.

Multimodal: Images, Audio, PDFs, and Beyond

Beyond text

Every modern frontier model can handle more than text. You can paste a screenshot, upload a PDF, attach an audio file, even share a short video. The model "sees" or "hears" it and can talk about what's in it.

How? The answer is elegant — and connects directly back to Part 1.

Everything becomes tokens

Recall from Part 1: text is split into tokens, each token becomes an ID, each ID is looked up in an embedding table to produce a row of meaning-numbers. The same idea works for images, audio, and other media — just with different tokenisation rules:



Images

Split into small **patches** (typically 14×14 or 16×16 pixels). Each patch is converted into a row of numbers by a **vision encoder**. Those rows become tokens in the context window.



Audio

Split into short **frames** (typically 25 milliseconds each). Each frame is converted into a row of numbers by an audio encoder. Those become tokens too.



PDFs

Usually a hybrid — text gets extracted and tokenised normally, and any embedded images or scanned pages are tokenised as images.

📌 **The deep idea:** Once everything is "a row of numbers in the context window," the model doesn't care where those numbers came from. Attention works the same way across words, image patches, and audio frames — all in the same sentence of thought.

A Concrete Example: The PDF Upload

An advisor uploads a client's quarterly statement (a PDF) and asks: *"What's the EM equity exposure on page 3?"*

PDF Parsing

The PDF is parsed. Text on each page is tokenised normally.

Vision Encoding

Page 3 contains a pie chart. The chart image is split into patches and tokenised by the vision encoder.

Context Assembly

All tokens — text from all pages, image patches from the chart — go into the context window. The user's question is added at the end.

Cross-Modal Attention

The model uses attention across **everything**: text and image patches alike. When attending to "page 3," it focuses on image tokens from that page.

Answer Generation

The model emits its answer — possibly with a tool call to verify the number against the portfolio system.

The Limits Worth Knowing

Images consume a lot of tokens

A single high-resolution image might be 1,000–3,000 tokens. A 20-page PDF with charts can easily eat 20,000+ tokens of your context window.

Vision is still weaker than text

Models often misread small numbers in tables, mistake similar logos, or hallucinate details in busy images. Don't trust extracted numbers without verification.

Audio and video are even more token-heavy

A one-minute audio clip might be 1,500 tokens. A one-minute video, far more.

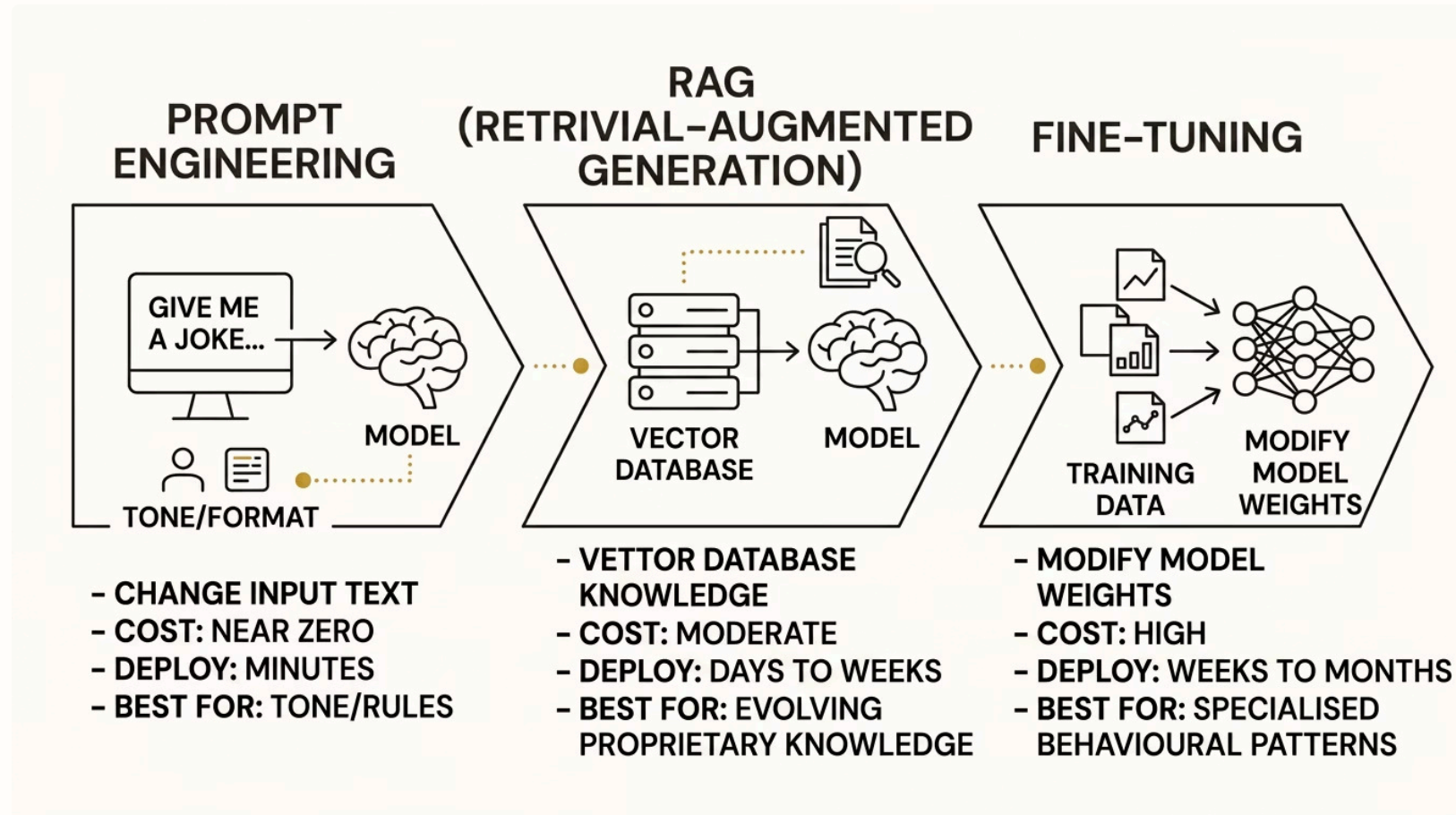
⚠ **The mental model:** When you upload anything, ask yourself *"How is this getting converted to tokens, and how many?"* The answer drives both cost and accuracy.



The Customisation Question: Prompt, RAG, or Fine-Tune?

This is the most common business question about LLMs. *"We have proprietary data and a specialised use case. How do we make the model good at our specific job?"*

There are three answers, and they work at very different layers, costs, and timescales.





Option 1: Prompt Engineering

What it is

Write a very good system prompt. Include examples of ideal outputs (called **few-shot examples**). Structure the inputs carefully. That's it — you change *what you say to the model*, not the model itself.

Key characteristics

- **Cost:** Almost zero — you're paying for the extra tokens in the prompt
- **Speed to deploy:** Minutes — edit a string
- **Best for:** Behaviour, tone, format, simple rules
- **Limits:** Can't add new knowledge; can't fix fundamental capability gaps



Example: Getting our wealth assistant to match the firm's tone of voice. *"Always round to 1 decimal. Never give tax advice. Respond in this JSON structure."* Pure prompt-engineering territory.

Option 2: RAG (Retrieval-Augmented Generation)

What it is

Put your knowledge into a vector database; retrieve relevant chunks per query; inject them into the prompt.

Key characteristics

- **Cost:** Moderate — pay for embedding documents once, storing them, and extra input tokens per query
- **Speed to deploy:** Days to weeks — you need a pipeline that indexes documents and stays fresh
- **Best for:** Letting the model use specific, evolving, or proprietary knowledge
- **Limits:** Quality depends entirely on retrieval; failure modes include bad chunking, stale indexes, misses

i Example: Making our assistant aware of the firm's house views, current product lineup, and compliance rules. Classic RAG territory.



Option 3: Fine-Tuning

What it is: Actually modify the model itself. Take a pre-trained model and run additional training on it using your own examples — usually thousands of input/output pairs showing the behaviour you want. The model's weights shift. The result is a new model, derived from the original but tailored to your task.

Cost

High. Thousands to hundreds of thousands of dollars per training run, depending on model size. Plus a higher per-token rate at inference time.

Speed to deploy

Weeks to months. You need a high-quality dataset, training infrastructure, and an evaluation process.

Best for

Specialised tasks where the *style* or *format* of output matters and is hard to specify in a prompt. Or where you need behaviour that even the best prompt can't reliably achieve.

Limits

Brittle to maintain. Fine-tuning doesn't reliably add facts — it shapes behaviour. Trying to teach facts via fine-tuning usually produces a model that hallucinates them confidently.

 **Example:** Training a model to draft regulatory filings in your firm's exact format and language — but probably *also* needs RAG for the current numbers.

A Decision Rule: The Right Order of Operations

1 Start with Prompt Engineering

Always. It's free, fast, and you'll be surprised how far it goes.

2 Add RAG

When the model needs knowledge it doesn't have, especially knowledge that changes over time.

3 Consider Fine-Tuning

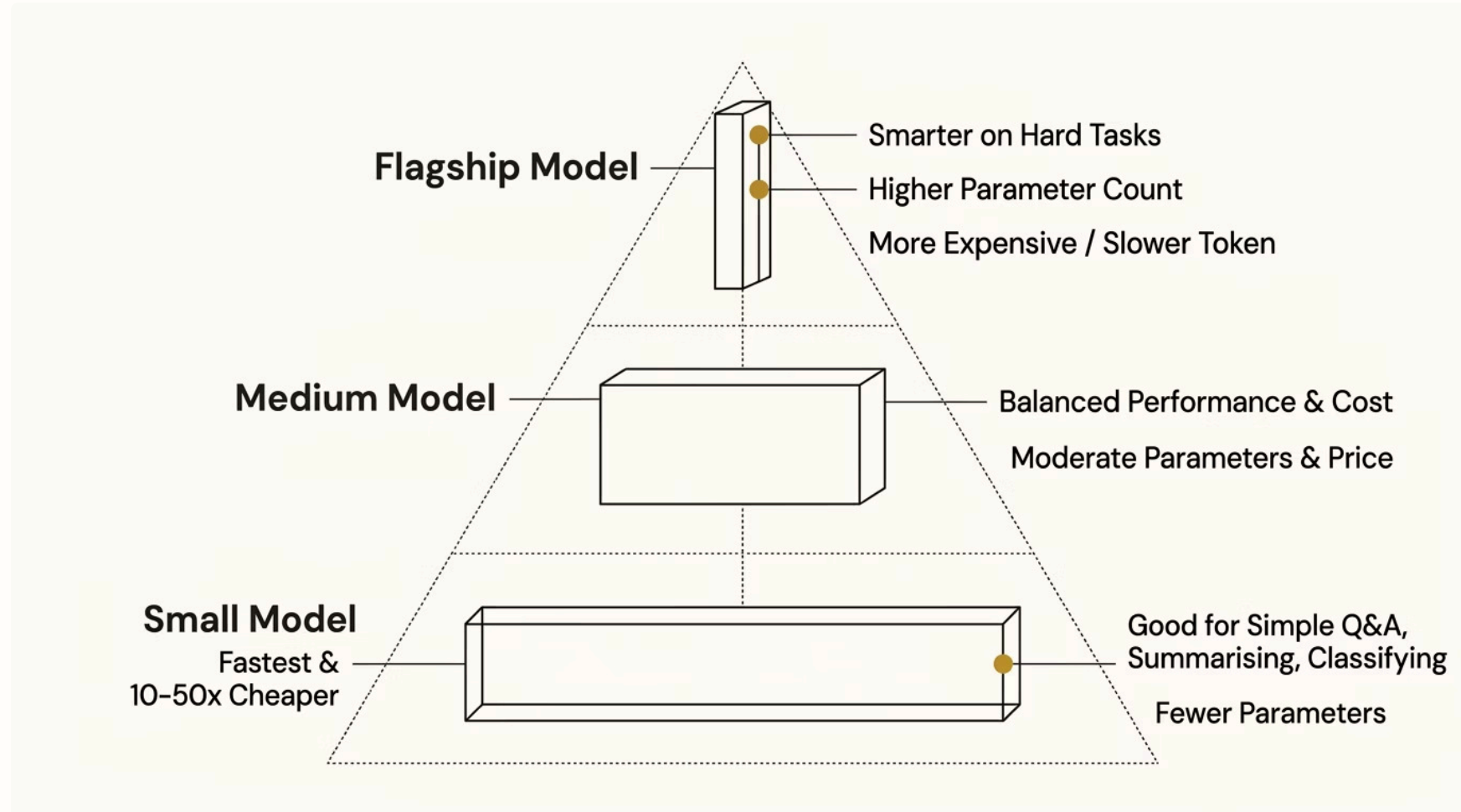
Only when prompting and RAG together aren't enough — usually because you need a very specific behavioural pattern and have a stable, high-quality dataset to train on.

 **The rookie mistake:** Reaching for fine-tuning first, because it sounds the most sophisticated. It's almost always the wrong starting point. Fine-tuning is the most expensive, least flexible, and most brittle option. In practice, most enterprise LLM systems are 90% prompting + RAG + tools, and 10% fine-tuning where it really pays off.

Model Size, Speed, and the Routing Game

Bigger isn't always better

When LLM vendors release new models, they typically release a family: a big flagship model, a medium model, and a small model. For example: Opus, Sonnet, Haiku. Or GPT-4, GPT-4-mini, GPT-4-nano.



How Smaller Models Exist: Distillation

A small model doesn't just appear from nowhere. It's typically created by **distillation** — training a small model to imitate the outputs of a larger model.

The Teacher

The large, powerful model. Generates high-quality outputs across a wide range of tasks. Acts as the source of truth during distillation training.

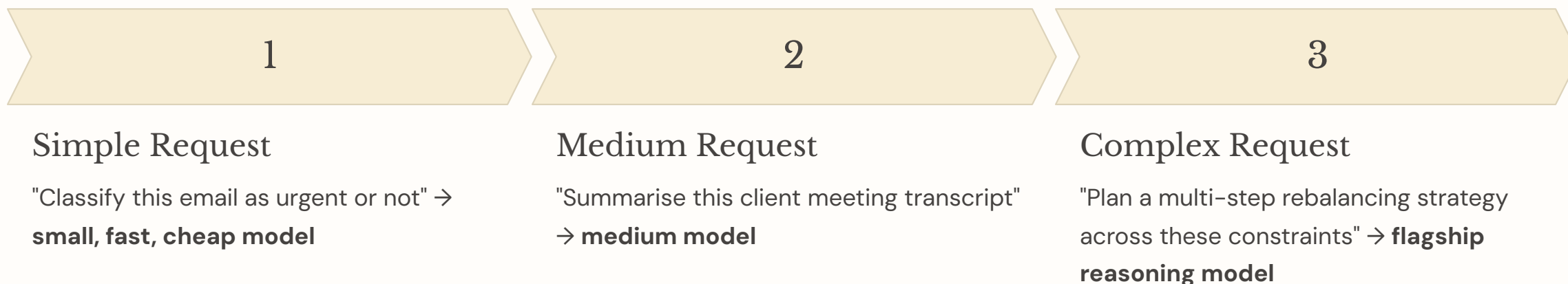
The Student

The small, fast model. Trained to approximate the teacher's outputs. Ends up much smaller and faster, but approximates the teacher's quality on most tasks. This is why model families feel coherent — the smaller members are derived from the larger ones, sharing much of the same training and personality.



The Routing Pattern

Production systems usually don't pick one model — they pick *the right model per request*. This is called **model routing**.

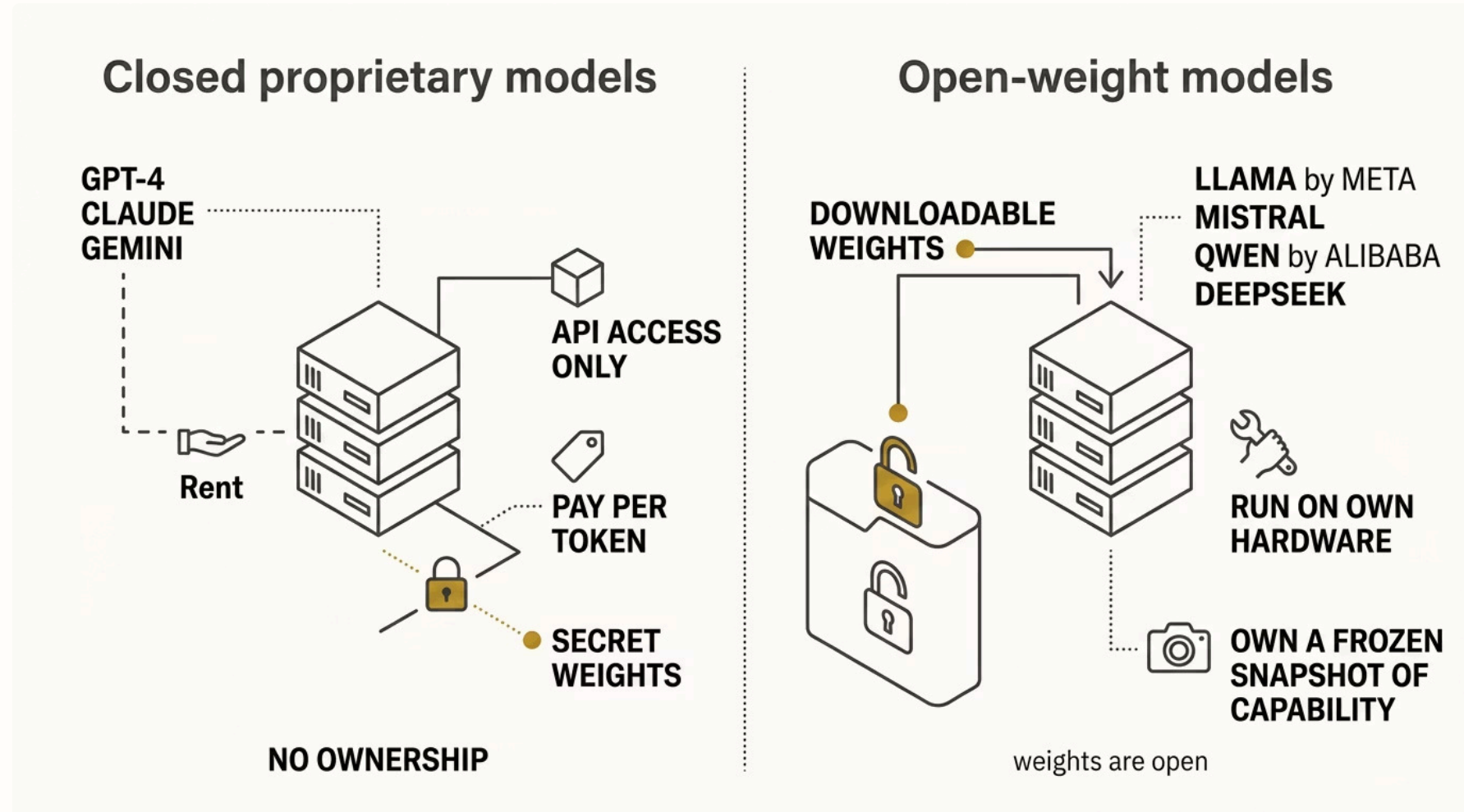


📌 **The lesson:** Model choice is a per-request decision, not a system-wide one. Picking the flagship for everything is wasteful. Picking the cheapest for everything is risky. The right answer is almost always a mix. The router itself is often another small LLM, or a simple classifier, that looks at the incoming request and decides which model to hand it to.

Open vs Closed Models

Two worlds

LLMs come in two broad categories that have very different strategic implications.



What Changes with Open-Weight Models

On-Premises Deployment

No client data leaves your network. Major appeal for finance, healthcare, defence, legal. For our wealth-management assistant, this can be the difference between "compliant" and "compliance nightmare."

Deep Fine-Tuning

With closed models, fine-tuning is limited to what the vendor permits. With open models, you have full control of the weights.

Operational Burden

Hosting an LLM is non-trivial. GPUs are expensive. Scaling, monitoring, patching, security — all on you. What was a billing relationship becomes an infrastructure project.

Quality Gap

As of 2026, the best open-weight models are competitive with mid-tier closed models on many tasks, but the absolute frontier is still in closed labs. This gap fluctuates.

✅ **The pragmatic answer:** Most enterprises end up with a hybrid — closed models for general-purpose assistant use, open models for specific high-volume, privacy-sensitive, or cost-sensitive workloads. It's rarely either/or.

Embeddings as a Standalone Product

A quick aside on something that's easy to miss. We've talked about embeddings throughout the series — the row of numbers that captures the meaning of a word, a sentence, or a document. So far, embeddings have appeared as a *step inside* the LLM pipeline.

But embeddings are also a **product in their own right**. Every major provider offers an embedding API: send text, get back its row of numbers. Use those numbers however you want — without ever invoking the full LLM.



Semantic Search

Find documents similar in *meaning* to a query, not just keyword matches. "*Client wants to reduce volatility*" matches "*low-risk portfolio rebalancing*" even though no words overlap.



Clustering

Group similar items together — emails by topic, clients by question type, research notes by theme.



Deduplication

Detect near-duplicate documents that aren't exactly identical.



Classification

Train a tiny classifier on top of embeddings to label things ("is this email about a complaint or a question?").



Recommendation

Suggest similar research notes, similar funds, similar past cases.



Why this matters: Embeddings are 10–100× cheaper than full LLM calls, and often dramatically faster. For many business problems framed as "*find the relevant thing*" rather than "*generate something new*," you don't need a chat model at all. You just need embeddings.

STEP 31

Privacy and Data Handling at the Provider Level

The question every enterprise asks

Before adopting any LLM, every serious organisation asks the same question:

"What does the provider do with our data?"

This is bigger than the cross-conversation isolation covered in Part 1, Slide 24 — that was about whether *your own* conversations leak across each other. This is about whether your data leaves your control at all.

Three Things Providers Do (or Don't) with Your Data

1

Training

Does the provider use your prompts and responses to train future model versions?

- **Free consumer tiers** — by default, yes. You can usually opt out in settings.
- **Paid consumer tiers** — varies by provider. Often *no* by default for paid plans.
- **API and enterprise tiers** — almost universally *no*. Providers contractually commit not to train on API data.

2

Retention

How long does the provider store your data, and who can access it?

- Most providers retain API requests for 30 days for abuse monitoring, then delete.
- **Zero-retention agreements (ZDR)** are available to enterprise customers.
- Storage location matters — some enterprises require data to stay in specific geographies.

3

Access

Who at the provider can see your data, under what circumstances?

- Standard answer: nobody, except for narrow abuse review processes.
- Subpoenas and legal requests are a separate issue and vary by jurisdiction.

What This Means in Practice

For a wealth-management assistant handling real client data, the requirements are typically:



API or Enterprise Plan

Never the free consumer tier.



No Training on Inputs

Contractually confirmed.



Zero or Short Retention

Confirmed in writing.



Data Residency

Matching client regulations.



Audit Logs

The firm controls them.



On-Prem or VPC Deployment

Model runs in infrastructure the firm controls; no data crosses the provider's network.

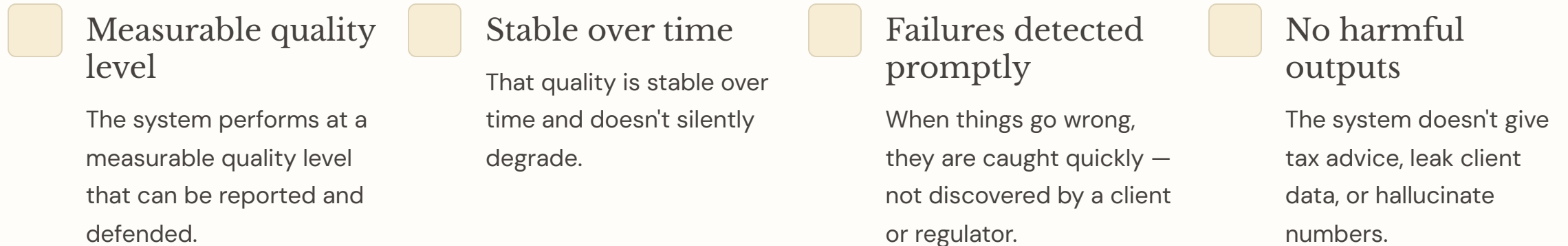


The lesson: "Does Anthropic / OpenAI / Google train on our data?" is the wrong shape of question. The right shape is *"Under which specific tier, contract, and deployment are we using their service, and what are the precise data-handling guarantees of that arrangement?"*

Evaluation: How Do You Know It's Working?

The compliance problem

For a wealth-management assistant, *"the model seems to give good answers"* is not a defensible position. Regulators, internal audit, and your own risk team will want evidence that:



This evidence comes from a discipline called **evaluation** (or just **evals**). Every serious LLM deployment needs it. Most don't have nearly enough.

The Four Layers of Evaluation

1

Automated Benchmarks

Standard tests like MMLU (general knowledge), HumanEval (code), GSM8K (math). Useful for picking a base model. Not very useful for your specific use case.

2

Task-Specific Test Sets

A handpicked set of representative questions for your actual use case, each with an expected answer or quality rubric. For our wealth assistant, this might be 200–500 real (anonymised) advisor questions with expert-validated correct answers.

3

LLM-as-Judge

Use a second LLM to evaluate the answers of the first. Cheaper than human review, can be run continuously. Imperfect — judges have their own biases — but valuable as an early warning system.

4

Human Review

For sensitive or high-stakes use cases, qualified humans (compliance officers, senior advisors) review a sample of real responses. Slow, expensive, but the only way to catch subtle failures that automated tools miss.

What Evaluation Actually Catches

Regressions

You changed the system prompt to fix one issue, and accidentally broke five others. Without an eval set, you'd never know.

Drift

The model behind your API changed (vendors update models silently). Quality shifted overnight. Evals catch this.

Edge Cases

The system is great 95% of the time but catastrophically wrong on a specific class of question — say, anything involving currency conversion. Evals expose this.

Compliance Violations

The model gave tax advice 3% of the time despite the system prompt forbidding it. Audit trail caught it.

⊗ **The uncomfortable truth:** Most production LLM systems run with almost no evaluation. Teams release changes based on whether things *feel* better, not whether they measurably are. For consumer products, this is sometimes okay. For regulated industries handling real money, it isn't.

A Note on What's Still Hard

After 30+ steps across four parts, you've seen the engineering reality of how these systems work. Worth a sober note on what they still struggle with — because no part of this series will fully prepare you for the gap between *what LLMs do well* and *what people assume they do well*.

Arithmetic

They predict digits as patterns, they don't compute. Always pair with tools for any real calculation.

Counting

"How many times does the letter R appear in 'strawberry'?" Famously hard for LLMs because of tokenisation. Don't trust their counts.

Knowing what they don't know

Models are bad at calibrating their own confidence. They sound equally certain about things they're right about and things they're making up.

Long-horizon planning

Reasoning models help, but multi-step plans with many constraints still trip them up. They can lose track of constraints over long contexts.

Anything time-sensitive

Cutoff dates. Need tools or RAG.

Consistency under rephrasing

Ask the same question two slightly different ways and you can get meaningfully different answers. This is a known failure of probability-based generation.

📌 **The shape of the right relationship:** Use them where they're strong, scaffold them where they're weak, verify where it matters. Trust nothing important without a check.

Bringing All Four Parts Together

From the second you press Enter, to the strategic questions a CTO has to answer — here is the full picture.

Part 1 — Delivery

How your text travels from your keyboard to a GPU, gets cleaned, bundled with context, tokenised, embedded, and positioned. The mechanical journey of a request.

Part 3 — The Real Assistant

Instruction tuning. System prompts. Tool use. Agentic loops. RAG. Guardrails. Stop conditions. Context-window management. The KV cache. Cost mechanics and prompt caching. The architecture of a working system.

1

2

3

4

Part 2 — Thinking

How the model processes those tokens. Attention. Next-word prediction. Probability distributions. Sampling (temperature, top-K, top-P). Detokenization. Streaming. The mechanics of generation.

Part 4 — Decisions

Knowledge cutoffs. Reasoning models. Multimodal inputs. The prompt-vs-RAG-vs-fine-tune decision. Model size and routing. Open vs closed models. Embeddings. Privacy and data handling. Evaluation. The honest limits.